Rapport SSIS — Alimentation du Data Warehouse RH

Oct 15, 2026 · @Ahmed Zekri

1. Résumé

Le package `Load_data_warehouse.dtsx` charge les 17 tables du schéma `dw` à partir des 7 tables de staging, et deux exécutions successives donnent exactement le même résultat (2 740 employés, 272 998 pointages, 18 679 inscriptions).

Attention : les deux connexions pointent aujourd'hui sur le serveur local, pas sur `tcp:100.102.67.72,61301`. Les deux exécutions ont donc lu `RH_Staging_Area` et écrit `RH_Data_Warehouse` sur l'instance SQL Server du poste `DESKTOP-4VHN986` (adresse Tailscale de ce poste : 100.85.125.78, donc un autre serveur). Les bases du serveur Tailscale n'ont pas été modifiées.

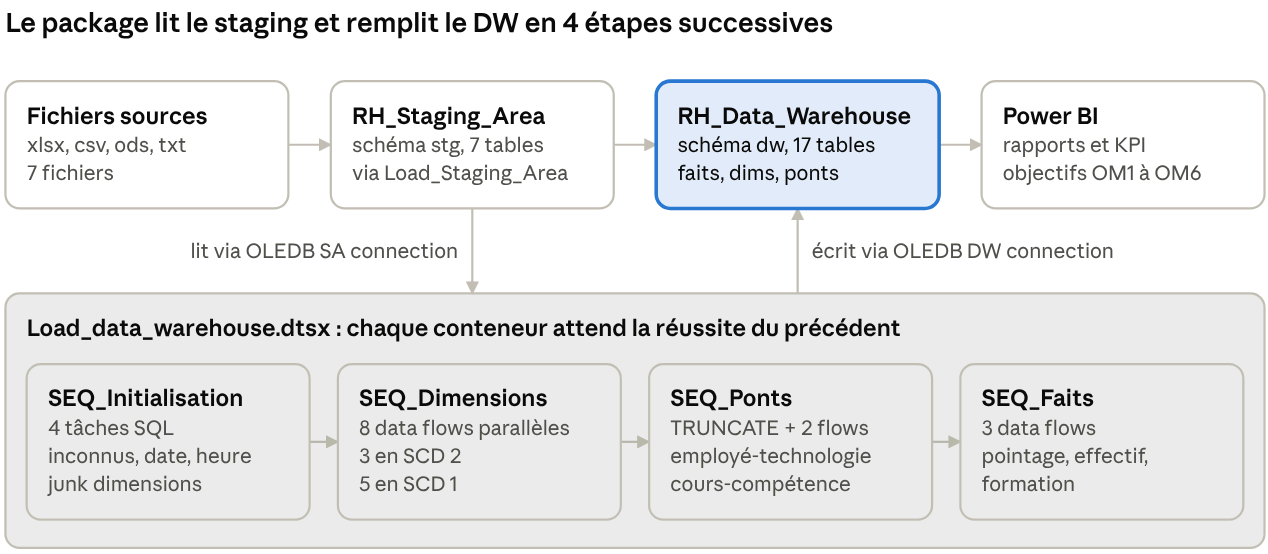
Mise à jour du 15/10/2026, 20:42 : les deux connexions du package ont depuis été basculées sur `tcp:100.102.67.72,61301` (login `BI`). Ce sont ces connexions qui sont publiées sur Git (section 12). Les volumes de la section 9 restent ceux mesurés sur la base locale.

Connection manager	Rôle	Chaîne actuelle	Serveur visé par le projet
`OLEDB SA connection`	Source : zone de staging	`Data Source=localhost; Initial Catalog=RH_Staging_Area; Integrated Security=SSPI`	`tcp:100.102.67.72,61301`, login `BI`
`OLEDB DW connection`	Cible : Data Warehouse (et références des Lookups)	`Data Source=localhost; Initial Catalog=RH_Data_Warehouse; Integrated Security=SSPI`	`tcp:100.102.67.72,61301`, login `BI`

La section 11 explique comment basculer sur le serveur Tailscale. Le package `Load_Staging_Area.dtsx`, lui, est déjà configuré sur ce serveur.

2. Architecture générale

Le projet comporte deux packages : `Load_Staging_Area.dtsx` copie les fichiers dans le staging, puis `Load_data_warehouse.dtsx` transforme le staging en modèle en étoile, en 4 conteneurs exécutés l'un après l'autre.



architecture du chargement · 4 conteneurs

L'ordre suit les dépendances. Les faits ont besoin des clés des dimensions et les ponts des employés, des cours et des compétences ; tout le monde a besoin des membres « Inconnu » et du calendrier. À l'intérieur d'un conteneur, les data flows sans dépendance entre eux tournent en parallèle.

Élément du package	Détail
Connection managers	`OLEDB SA connection` (lecture du staging) et `OLEDB DW connection` (écriture du DW et tables de référence des Lookups)
Variables	`SnapshotDateOverride`, `SnapshotDate`, `SnapshotDateKey` (section 8)
Volume	4 conteneurs, 8 tâches SQL, 13 data flows
Protection	`EncryptSensitiveWithUserKey`, comme le projet

3. Notions SSIS à connaître

Chaque composant du package appartient à l'une des catégories ci-dessous ; si on les comprend, on comprend tout le package.

Les deux niveaux d'un package

- **Control Flow (flux de contrôle)** : l'ordre d'exécution des tâches. Les flèches vertes sont des *contraintes de précédence* « Success » : la tâche suivante ne démarre que si la précédente a réussi. Deux tâches sans flèche entre elles s'exécutent en parallèle.
- **Data Flow (flux de données)** : à l'intérieur d'une tâche *Data Flow Task* (préfixe `DFT_`), les lignes circulent en mémoire, par paquets (buffers), d'une source vers une destination en passant par des transformations.

Tâches du Control Flow

Composant	Préfixe	Ce qu'il fait
Sequence Container	`SEQ_`	Regroupe des tâches. Le conteneur suivant attend que toutes les tâches du conteneur aient réussi.
Execute SQL Task	`SQL_`	Envoie une requête SQL au serveur (création de lignes, TRUNCATE, DELETE). Aucune ligne ne passe par SSIS.
Data Flow Task	`DFT_`	Contient un pipeline source → transformations → destination.

Composants des Data Flows

Composant	Préfixe	Ce qu'il fait
OLE DB Source	`SRC_`	Exécute une requête SELECT sur le staging et produit les lignes. Le nettoyage (conversion de types, TRIM, normalisation) est fait dans cette requête.

Composant	Préfixe	Ce qu'il fait
Lookup	`LKP_`	Cherche chaque ligne dans une table de référence du DW (ex. trouver `employe_sk` à partir de `employe_bk`). Mode « Full cache » : la référence est chargée une fois en mémoire. Sortie *Match* = trouvé, sortie *No Match* = pas trouvé.
Derived Column	`DER_`	Ajoute des colonnes calculées par une expression SSIS (dates SCD, clés -1, coûts, date de chargement).
Conditional Split	`CSPL_`	Aiguille les lignes selon une condition (ici : l'empreinte a-t-elle changé ?). Les lignes qui ne vérifient aucune condition vont dans la sortie par défaut, non reliée, donc ignorées.
Multicast	`MC_`	Duplique chaque ligne vers plusieurs sorties (ici : une copie pour clôturer l'ancienne version, une pour insérer la nouvelle).
OLE DB Command	`CMD_`	Exécute une requête UPDATE pour chaque ligne reçue, avec des paramètres `?`. Lent sur de gros volumes, mais ici il ne traite que les lignes modifiées.
OLE DB Destination	`DST_`	Insère les lignes dans une table du DW en *fast load* (insertion en masse). Chaque colonne de la table reçoit la colonne amont de même nom.

Notions de modélisation utilisées

- **Clé métier (`_bk`)** : l'identifiant venant de la source (ex. l'UUID de l'employé). **Clé de substitution (`_sk`)** : un entier généré par le DW (colonne IDENTITY). Les faits ne stockent que des `_sk`.
- **SCD 1** (Slowly Changing Dimension type 1) : si un attribut change, on écrase l'ancienne valeur. Pas d'historique.
- **SCD 2** : si un attribut change, on garde l'ancienne ligne (clôturée avec `date_fin` et `is_current = 0`) et on insère une nouvelle ligne avec une nouvelle `_sk`. L'historique est conservé.

- **`row_hash`** : empreinte SHA-256 de tous les attributs suivis d'une ligne. Comparer deux empreintes revient à comparer toutes les colonnes d'un coup.
- **Membre « Inconnu » (`_sk = -1`)** : une ligne spéciale dans chaque dimension. Un fait dont la dimension est introuvable pointe vers -1 au lieu d'être perdu ou de violer une clé étrangère.
- **Junk dimension** : une dimension qui regroupe des indicateurs oui/non (flags). On génère toutes les combinaisons à l'avance ; le fait pointe vers la combinaison qui le concerne.
- **Table de pont (bridge)** : résout une relation plusieurs-à-plusieurs (un employé a plusieurs technologies, une technologie concerne plusieurs employés).
- **Snapshot périodique** : un fait qui photographie un état à une date (l'effectif au 15/10/2026). Ses mesures ne s'additionnent pas d'une date à l'autre.

4. SEQ_Initialisation : les tables qui ne viennent pas des fichiers

Ce conteneur prépare tout ce que les autres étapes supposent déjà présent : les membres « Inconnu », le calendrier, les heures et les combinaisons de flags. Ses 4 tâches *Execute SQL* s'exécutent l'une après l'autre sur la connexion DW.

Toutes sont **idempotentes** : chaque INSERT est filtré par `WHERE NOT EXISTS` (ou `IF NOT EXISTS`), donc relancer le package n'ajoute aucune ligne. C'est ce qu'on a vérifié à la 2e exécution.

Tâche	Table(s)	Ce qu'elle fait	Résultat
`SQL_Init_Membres_Inconnus`	Les 12 dimensions	Insère la ligne `_sk = -1` (« Inconnu ») dans chaque dimension si elle manque. Pour les colonnes IDENTITY, la tâche active `SET IDENTITY_INSERT ON` le temps de forcer la valeur -1.	1 ligne -1 par dimension (la date -1 et l'heure -1 aussi)

Tâche	Table(s)	Ce qu'elle fait	Résultat
`SQL_Generer_Dim_Date`	`dim_date`	Génère un jour par ligne du 01/01/1950 au 31/12/2040. Calcule le jour de la semaine (1 = lundi), la semaine ISO, le mois et le trimestre en français, `annee_mois` (AAAA-MM), les fériés et les jours ouvrés.	33 238 dates + la ligne -1
`SQL_Generer_Dim_Heure`	`dim_heure`	Génère une ligne par minute de la journée. La clé vaut HHMM (8 h 30 → 830) et chaque minute est rangée dans sa tranche de 15 minutes (`08:30-08:45`).	1 440 minutes + la ligne -1
`SQL_Generer_Junk_Dimensions`	`dim_flags_pointage` , `dim_flags_effectif`	Génère toutes les combinaisons de flags par produit cartésien (`CROSS JOIN`) de tables {0, 1}.	32 combinaisons (2^5) ; 24 combinaisons (3 statuts × 2^3)

Pourquoi 1950–2040 ? Les dates d'embauche commencent en 2000, les fins de contrat vont jusqu'en 2024 et les snapshots sont datés du jour du chargement : la plage couvre tout avec de la marge. Une date hors plage reçoit la clé -1.

Jours fériés : ce sont les fêtes nationales tunisiennes à date fixe : 1er janvier, 20 mars, 9 avril, 1er mai, 25 juillet, 13 août et 15 octobre. Le 14 janvier compte de 2012 à 2020, puis le 17 décembre à partir de 2021. Les fêtes religieuses (Aïd, Mouled, Ras El Am Hejri) changent de date chaque année et ne sont pas gérées. Un jour est ouvré s'il tombe du lundi au vendredi et n'est pas férié ; 2023 compte 257 jours ouvrés.

Pourquoi générer les junk dimensions à l'avance ? Le fait n'a plus qu'à chercher sa combinaison avec un Lookup. Sans cela, il faudrait insérer les nouvelles combinaisons pendant le chargement des faits. Dans `dim_flags_effectif` , `is_anomalie_contrat` n'est pas un choix libre : il vaut 1 dès qu'un des trois autres flags vaut 1.

5. SEQ_Dimensions : 8 data flows en parallèle

Les 8 dimensions issues du staging sont chargées en même temps, car aucune ne dépend d'une autre. Trois suivent le modèle SCD 2 (historique), cinq le modèle SCD 1.

5.1 Le modèle SCD 2 : DFT_DIM_EMPLOYE, DFT_DIM_DEPARTEMENT, DFT_DIM_TAUX_HORAIRE

Les trois data flows ont exactement la même structure ; seuls la requête source, la table et la clé changent.

Data flow	Requête source (staging)	Clé métier	Attributs suivis par le `row_hash`
`DFT_DIM_EMPLOYE`	`emp_data` (population « Technique ») UNION ALL `administration_data` (« Administratif »)	`employe_bk` = `CODE` / `ID_Employee`	population, matricule, nom, genre, adresses, statut marital, e-mail, langue, université, diplôme, canal de recrutement, technologies
`DFT_DIM_DEPARTEMENT`	`departments`	`departement_bk` = `ID_DEP`	nom, capacité, pôle (`DEP_ADMIN00` → Administratif, sinon Technique)
`DFT_DIM_TAUX_HORAIRE`	`extrainfos`	`code_taux` déduit du libellé : `VACATAIRE` (35), `HS_PERMANENT` (25), `HS_ADMIN` (22)	libellé, valeur du taux

Les composants, dans l'ordre :

1. **`SRC_dim_xxx` (OLE DB Source)** : lit et nettoie le staging, puis calcule `row_hash = HASHBYTES('SHA2_256', CONCAT(attribut1, '|', attribut2, …))`. Les NULL sont remplacés par `~` avant le hachage pour que « NULL » et « vide » ne donnent pas la même empreinte. La colonne `row_hash_hex` est la même empreinte en texte, plus simple à comparer dans SSIS.
2. **`LKP_Version_Courante` (Lookup)** : cherche la clé métier parmi les versions courantes du DW (`WHERE is_current = 1`) et récupère `lkp_sk` (la clé de la version courante) et `lkp_hash` (son empreinte). Elle est configurée en « Redirect rows to no match output ».
3. **Sortie No Match → `DER_Nouvelle_Cle` → `DST_Nouvelle_Cle`** : l'employé n'existe pas encore. Le Derived Column fixe `date_debut = 1900-01-01`, `date_fin = 9999-12-31`, `is_current = 1` et `etl_load_date` ; la destination insère la ligne, et SQL Server attribue la `_sk`.
4. **Sortie Match → `CSPL_Changement`** : l'employé existe. La condition `ISNULL(lkp_hash) || row_hash_hex != lkp_hash` envoie dans la sortie « Modifie » seulement les lignes dont un attribut a changé. Les lignes inchangées partent dans la sortie par défaut, non reliée : on ne fait rien.
5. **`DER_Nouvelle_Version`** : prépare la nouvelle version avec `date_debut` = date de snapshot, `date_fin = 9999-12-31` et `is_current = 1`. Il calcule aussi `date_fin_ancienne` = la veille de la date de snapshot.
6. **`MC_Version` (Multicast)** : duplique chaque ligne modifiée en deux.
7. **`CMD_Cloturer_Ancienne_Version` (OLE DB Command)** : `UPDATE dw.dim_xxx SET date_fin = ?, is_current = 0 WHERE xxx_sk = ?`, avec `date_fin_ancienne` et `lkp_sk` comme paramètres. L'ancienne version est fermée.
8. **`DST_Nouvelle_Version`** : insère la nouvelle version, qui reçoit une nouvelle `_sk`.

Pourquoi `date_debut = 1900-01-01` pour une première version ? Elle est alors valable « depuis toujours » : un pointage de 2023 retrouve donc une version valide même si le DW n'a été chargé qu'en 2026. Seules les versions créées par un changement commencent à la date de snapshot.

Résultat : 2 740 employés, 4 départements, 3 taux, et 0 version clôturée après la 2e exécution, ce qui est attendu puisque rien n'a changé dans la source.

5.2 Le modèle SCD 1 sans mise à jour : contrat, poste, type de congé, compétence

Dans ces quatre dimensions, la combinaison d'attributs **est** l'identité du membre : un contrat « CDD / Vacataire » ne peut pas « changer », ce serait un autre membre. Il n'y a donc rien à mettre à jour, seulement les nouvelles combinaisons à insérer.

Le flux comporte quatre composants : `SRC_` (valeurs distinctes du staging) → `LKP_` (redirect no match) → sortie No Match → `DER_Audit` (ajoute `etl_load_date`) → `DST_`. Les lignes trouvées (sortie Match non reliée) existent déjà et sont ignorées.

Data flow	Source	Clé de recherche	Lignes
`DFT_DIM_CONTRAT`	Effectif technique + administratif	`type_contrat` + `position` ; `is_contrat_court = 1` pour CDD et CIVP	10 + Inconnu
`DFT_DIM_POSTE`	idem	`grade` + `rang` + `activite` ; pour les administratifs, rang et activité = « Non applicable »	1 417 + Inconnu, Non applicable
`DFT_DIM_TYPE_CONGE`	`LEAVE_TYPE` et `LAST_LEAVE_TYPE`, en minuscules	`type_conge` (mission, ordinary, sick, sans_solde)	4 + Inconnu
`DFT_DIM_COMPETENCE`	Technologies des employés + compétences des cours	`competence_bk` = libellé en minuscules	8 554 + Inconnu

Les requêtes de contrat, poste et type de congé réutilisent **exactement** le même bloc SQL que le fait `fact_effectif_snapshot`. Ainsi, les valeurs cherchées par le fait sont forcément celles qui ont été insérées dans la dimension.

5.3 Le modèle SCD 1 avec mise à jour : DFT_DIM_COURS

Un cours a une clé métier stable (`course_id`) et des attributs qui peuvent évoluer (note, description, etc.), qu'on écrase sans historique.

1. `SRC_dim_cours` : catalogue Coursera, une ligne par `course_id` (les 10 lignes sans identifiant sont écartées). La note « Not Calibrated » devient NULL. Une empreinte est calculée sur tous les attributs.

2. `LKP_dim_cours` : recalcule la même empreinte côté DW, directement dans sa requête de référence.
3. No Match → `DER_Audit` → `DST_dim_cours` : insertion des nouveaux cours.
4. Match → `CSPL_Changement` (empreintes différentes) → `CMD_Mettre_A_Jour` : `UPDATE dw.dim_cours SET … WHERE cours_sk = ?`. C'est l'écrasement typique du SCD 1.

Résultat : 3 522 cours + Inconnu, et aucune mise à jour à la 2e exécution.

6. SEQ_Ponts : relier employés, cours et compétences

Les deux tables de pont transforment des listes stockées dans une seule cellule en couples de clés : 2 197 couples employé-technologie et 35 232 couples cours-compétence.

`SQL_Truncate_Bridges` vide d'abord les deux tables (`TRUNCATE TABLE`), puis les deux data flows s'exécutent en parallèle. On recharge tout à chaque exécution, car un pont n'a pas d'historique et doit toujours pointer vers les versions courantes des employés.

L'éclatement des listes (dans la requête source)

Colonne source	Exemple de valeur	Séparateur	Fonction SQL
`emp_data.TECHNOLOGIES`	`Software Engineering, Telecom`	virgule	`STRING_SPLIT(..., ',')`
`coursera_trainings.Skills`	`Finance business plan Planning`	double espace	double espace remplacé par un caractère invisible (`NCHAR(31)`), puis `STRING_SPLIT` sur ce caractère (`STRING_SPLIT` n'accepte qu'un séparateur d'un seul caractère)

Chaque morceau est nettoyé (TRIM, retours à la ligne retirés), mis en minuscules pour former `competence_bk` , puis dédoublonné par `GROUP BY` . Le même bloc SQL (`_competences`) alimente aussi `dim_competence` : technologies et compétences des

cours partagent donc un seul référentiel. « Telecom » côté employé et « telecom » côté cours donnent la même compétence.

Les composants de chaque data flow

1. **`SRC_bridge_xxx`** : renvoie les couples (clé métier, `competence_bk`) avec les colonnes d'audit.
2. **`LKP_Employe`** ou **`LKP_Cours`** : remplace la clé métier par la `_sk` (version courante pour l'employé).
3. **`LKP_Competence`** : remplace `competence_bk` par `competence_sk`.
4. **`DER_Audit`** : ajoute `etl_load_date`.
5. **`DST_bridge_xxx`** : insertion en masse (TABLOCK).

Ici, les Lookups sont en « redirect no match » avec une sortie No Match non reliée : un couple dont une clé est introuvable est **écarté** au lieu d'aller vers -1. Avec -1, deux compétences inconnues d'un même employé produiraient deux fois le couple (employé, -1), ce qui violerait la clé primaire du pont. Dans les faits, aucun couple n'a été écarté : les volumes sont identiques à ceux de la source.

Limite connue : dans `Skills`, les derniers mots-clés d'un cours sont parfois séparés par un seul espace (ex. `unix shells arts-and-humanities`). Ils restent alors dans un seul morceau, d'où quelques « compétences » longues dans `dim_competence`.

7. SEQ_Faits : les trois tables de faits

Les trois faits sont chargés en parallèle, chacun précédé d'une tâche SQL qui retire les données d'une exécution précédente. C'est ce qui rend le package relançable sans doublon.

Fait	Grain (une ligne =)	Tâche préalable	Pourquoi
`fact_pointage`	1 pointage d'un employé un jour	`SQL_Truncate_Fact_Pointage` : `TRUNCATE TABLE`	Le staging contient tout l'historique 2023, on recharge tout
`fact_effectif_snapshot`	1 employé à 1 date de snapshot	`SQL_Delete_Snapshot_Effectif` : `DELETE … WHERE snapshot_date_key = <date du jour>`	Seule la photo du jour est remplacée ; les photos des jours précédents sont conservées
`fact_formation`	1 employé × 1 cours à 1 date de snapshot	`SQL_Delete_Snapshot_Formation` : même principe	idem

Le texte des deux DELETE est construit par une **expression SSIS** sur la propriété `SqlStatementSource`, qui y concatène la variable `SnapshotDateKey`.

Dans les trois flux, les Lookups sont en mode **« Ignore failure »** : une ligne sans correspondance continue avec une clé NULL, puis le Derived Column final remplace ce NULL par -1 (`ISNULL([lkp_employe_sk]) ? -1 : [lkp_employe_sk]`). Aucun fait n'est perdu, et chaque clé étrangère reste valide.

7.1 DFT_FACT_POINTAGE (272 998 lignes)

1. **`SRC_fact_pointage`** : `attendance_tech_2023` UNION ALL `attendance_admin_2023`. La requête :

- convertit les dates (format M/J/AAAA côté technique, AAAA-MM-JJ côté admin) et les heures (`H:MM:SS` ou `HH:MM`) en types `date` et `time` ;
- calcule `date_key` (AAAAMMJJ) et `heure_debut_key` / `heure_fin_key` (HHMM, -1 si l'heure est illisible) ;
- calcule les mesures : `minutes_pause` = reprise − pause ; `minutes_travaillees` = (fin − début) − pause ; `heures_travaillees` = minutes / 60 ; `heures_sup` = heures au-delà de 8 h ; `minutes_retard` = minutes après 08:30 ;
- calcule les 5 flags : retard, heures sup, journée de moins d'1 h, durée négative, heure illisible ;
- détermine le département (fiche employé pour les techniques, `DEP_ADMIN00` pour l'administration) et le code du taux des heures sup : `VACATAIRE` pour un vacataire, `HS_PERMANENT` pour un permanent, `HS_ADMIN` pour l'administration.

2. **`LKP_Employe`** : `employe_bk` → `employe_sk` (version courante).
3. **`LKP_Departement`** : `departement_bk` → `departement_sk` .
4. **`LKP_Flags_Pointage`** : les 5 flags → `flags_pointage_sk` (la combinaison dans la junk dimension).
5. **`LKP_Taux_HS`** : `code_taux_hs` → `taux_sk` et `valeur_taux` .
6. **`DER_Cles_Et_Mesures`** : clés NULL → -1, `cout_heures_sup = heures_sup × valeur_taux` , `nb_pointages = 1` , `etl_load_date` .
7. **`DST_fact_pointage`** : insertion en masse ; `pointage_sk` est généré par IDENTITY.

Les heures saisies (`heure_debut` , `heure_pause` , `heure_reprise` , `heure_fin`) sont stockées telles quelles, en plus des clés vers `dim_heure` , comme le prévoit le dossier de conception.

7.2 DFT_FACT_EFFECTIF_SNAPSHOT (2 740 lignes par snapshot)

1. **`SRC_fact_effectif_snapshot`** : effectif technique + administratif. La requête convertit salaire, heures, expérience et congés (certains sont en notation scientifique, ex. `4.38E-2`). Elle calcule les clés des 3 dates jouant un rôle (embauche, début et fin de contrat), les codes de taux (`VACATAIRE` seulement pour les vacataires, `HS_ADMIN` seulement pour les administratifs) et deux flags : `is_cdi_avec_date_fin` et `is_vacataire_en_cdi` .
2. **`DER_Indicateurs_Snapshot`** : calcule ce qui dépend de la date de snapshot :
   - `is_contrat_echu_actif` : date de fin de contrat dépassée alors que l'employé figure toujours dans l'effectif ;

- `is_anomalie_contrat` : au moins un des trois flags d'anomalie ;
- `anciennete_jours` : jours entre l'embauche et le snapshot ;
- `snapshot_date_key`.

3. **8 Lookups en chaîne** : `LKP_Employe`, `LKP_Departement`, `LKP_Contrat` (type + position), `LKP_Poste` (grade + rang + activité), `LKP_Type_Conge`, `LKP_Flags_Effectif` (statut + 3 flags), `LKP_Taux_Vacataire` et `LKP_Taux_HS_Admin` (clé + valeur du taux).
4. **`DER_Cles_Et_Mesures`** : clés NULL → -1, `nb_employes = 1`, et les coûts :
   - `cout_vacataire` = heures payées × 35 ;
   - `cout_hs_admin` = heures sup admin × 22 ;
   - `cout_total_estime` = salaire + coût vacataire + coût HS admin, chaque terme NULL comptant pour 0. C'est la formule de coût de l'EDA (OM2).
5. **`DST_fact_effectif_snapshot`**.

Les deux clés de taux restent **NULL** quand elles ne s'appliquent pas, par exemple pas de taux vacataire pour un permanent : le dossier de conception les a définies comme facultatives.

7.3 DFT_FACT_FORMATION (18 679 lignes par snapshot)

1. **`SRC_fact_formation`** : `emp_data.ID_TRAINING` contient une liste JSON (`[2466, 109, …]`). `OPENJSON` la décompose en une ligne par cours. Seuls les identifiants présents dans le catalogue sont gardés, ce qui écarte 6 inscriptions sur 18 685.
2. **`LKP_Employe`**, **`LKP_Cours`**, **`LKP_Departement`**.
3. **`DER_Cles_Et_Mesures`** : `snapshot_date_key`, clés -1, `nb_inscriptions = 1`, `etl_load_date`.
4. **`DST_fact_formation`**.

Colonnes d'audit (toutes les tables)

Colonne	Valeur
`etl_load_date`	Heure de début du package (`@[System::StartTime]`), identique pour toutes les lignes d'une exécution

Colonne	Valeur
`src_load_date`	`_LoadDate` de la ligne de staging (quand le staging a été chargé)
`src_file`	`_SourceFile` : fichier d'origine (ex. `emp_data.xlsx`)
`src_batch_id`	`_BatchId` du chargement du staging

8. Règles métier et nettoyage des données

Le staging stocke tout en texte (`nvarchar`) : le nettoyage se fait dans les requêtes `SRC_`, et les seuils métier reprennent ceux de l'EDA.

Nettoyage technique

Problème constaté dans le staging	Traitement
Tout est en `nvarchar`, y compris montants et dates	`TRY_CONVERT` vers le bon type ; une valeur non convertible devient NULL au lieu de faire échouer le chargement
Nombres en notation scientifique (`8.33E-2`)	Conversion en `float`, puis en `decimal`
Dates techniques au format `1/2/2023`, administratives au format `2023-01-02`	Style 101 (mois/jour/année) pour la source technique, style 23 (ISO) pour la source administrative
Heures `8:56:00` d'un côté, `08:44` de l'autre	`TRY_CONVERT(time(0), …)` accepte les deux formats
Espaces parasites	`LTRIM(RTRIM(…))` sur toutes les clés et tous les libellés
Casse incohérente (`Sick` / `sick`, `Free Application` / `free_application`)	Minuscules, et espaces remplacés par `_` pour le canal de recrutement
Ligne entièrement vide dans `administration_data`	Filtrée (`employe_bk` NULL)
`MARITAL_STATUS` = 0/1	1 → « Marié(e) », 0 → « Non marié(e) »

Problème constaté dans le staging	Traitement
`TENURE` = 0/1	1 → « Titularisé », 0 → « Non titularisé », sinon « Inconnu »
Note de cours « Not Calibrated » ou vide	NULL
Valeur manquante dans un attribut servant de clé (type de contrat, grade…)	« Inconnu », pour que le Lookup du fait la retrouve

Règles métier

Règle	Définition	Origine
Retard	Arrivée après 08:30 ; `minutes_retard` = minutes au-delà de 08:30	EDA, OM3
Heures supplémentaires	Temps travaillé > 8 h ; `heures_sup` = excédent	EDA, OM3
Temps travaillé	(fin − début) − (reprise − pause)	EDA, OM3
Anomalies de pointage	Durée négative ; journée de moins d'1 h ; au moins une heure illisible	Qualité des données
Contrat court	CDD ou CIVP	EDA, OM1
Coût vacataire	Heures payées × 35	`extrainfos.txt`
Coût des heures sup administratives	`extra_hour` × 22	`extrainfos.txt`
Coût des heures sup au pointage	Heures sup × taux (25 permanent, 35 vacataire, 22 administratif)	`extrainfos.txt`
Coût total estimé	Salaire + coût vacataire + coût HS admin	EDA, OM2
Anomalies de contrat	CDI avec date de fin ; contrat échu mais employé toujours présent ; vacataire en CDI	Dossier de conception
Pôle	`DEP_ADMIN00` → Administratif ; les autres départements → Technique	Dossier de conception

La date de snapshot est pilotée par trois variables du package :

Variable	Type	Rôle
`SnapshotDateOverride`	String	Vide par défaut. Si on y met `2023-12-31`, le snapshot est daté de ce jour-là
`SnapshotDate`	DateTime, calculée	Date du jour, ou la date forcée. Sert aussi de date de début des nouvelles versions SCD 2

Variable	Type	Rôle
`SnapshotDateKey`	Int32, calculée	`SnapshotDate` au format AAAAMMJJ, la clé vers `dim_date`

Avec un snapshot daté d'aujourd'hui (2026), tous les contrats qui ont une date de fin sont déjà échus, car ils se terminent au plus tard fin 2024. D'où 53 % d'anomalies de contrat (1 462 employés sur 2 740). Avec `SnapshotDateOverride = 2023-12-31`, l'indicateur reflète la situation à la fin de la période observée.

9. Résultats et contrôles de qualité

Après deux exécutions (20:16 puis 20:31 le 15/10/2026, base locale), chaque table contient exactement le nombre de lignes attendu d'après la source. La 2e exécution n'a créé ni doublon ni nouvelle version.

Table	Lignes	Attendu d'après la source	Après la 2e exécution
`dim_employe`	2 741	1 461 + 1 279 + Inconnu	Identique, 0 version clôturée
`dim_departement`	5	4 + Inconnu	Identique
`dim_taux_horaire`	4	3 + Inconnu	Identique
`dim_contrat`	11	10 combinaisons + Inconnu	Identique
`dim_poste`	1 419	1 417 combinaisons + Inconnu + Non applicable	Identique
`dim_type_conge`	5	4 + Inconnu	Identique
`dim_cours`	3 523	3 522 + Inconnu	Identique
`dim_competence`	8 555	8 554 + Inconnu	Identique
`dim_date`	33 239	1950–2040 + Inconnu	Identique
`dim_heure`	1 441	1 440 minutes + Inconnu	Identique
`dim_flags_pointage`	33	32 + Inconnu	Identique

Table	Lignes	Attendu d'après la source	Après la 2e exécution
`dim_flags_effectif`	25	24 + Inconnu	Identique
`bridge_employe_technologie`	2 197	2 197 couples	Rechargée, identique
`bridge_cours_competence`	35 232	35 232 couples	Rechargée, identique
`fact_pointage`	272 998	189 799 + 83 200 − 1 date illisible	Rechargée à 20:31, identique
`fact_effectif_snapshot`	2 740	1 snapshot (20261015)	Snapshot remplacé, pas dupliqué
`fact_formation`	18 679	18 685 − 6 cours absents du catalogue	Snapshot remplacé, pas dupliqué

Les clés -1 sont toutes expliquées par la source

Fait	Clé à -1	Lignes	Cause dans la source
`fact_pointage`	`employe_sk`	65	1 agent administratif pointe mais n'a pas de fiche dans `administration_data` (l'orphelin déjà repéré dans l'EDA)
`fact_pointage`	`departement_sk`	520	4 employés techniques ont un `DEP_ID` vide
`fact_pointage`	`heure_debut_key`	25	Heure d'arrivée illisible (flag `is_heure_illisible`)
`fact_effectif_snapshot`	`departement_sk`	4	Les mêmes 4 employés sans `DEP_ID`
`fact_effectif_snapshot`	`date_embauche_key`	7	`STARTING_DATE` vide pour 7 employés techniques

Fait	Clé à -1	Lignes	Cause dans la source
`fact_formation`	`departement_sk`	61	Formations des 4 employés sans `DEP_ID`

Contrôles de cohérence

- **SCD 2 :** chaque `employe_bk` n'a qu'une ligne, et elle est courante.
- **Pointage :** en moyenne 6,50 h travaillées par jour côté technique et 5,99 h côté administratif. Les retards concernent 49,1 % et 49,4 % des jours, soit les mêmes ordres de grandeur que l'EDA.
- **Coûts (snapshot) :** 4,71 M côté technique (masse salariale 2,08 M + vacataires 2,63 M) et 6,63 M côté administratif (salaires 3,77 M + heures sup 2,86 M). Les sources ne précisent pas la devise.
- **Accents :** stockés correctement (vérifié par le code Unicode de « ê » dans « Fête »).

Requête de contrôle à relancer après chaque chargement

```
SELECT s.name + '.' + t.name AS table_dw,
       SUM(p.rows)            AS nb_lignes
FROM sys.tables t
JOIN sys.schemas s    ON s.schema_id = t.schema_id
JOIN sys.partitions p ON p.object_id = t.object_id AND p.index_id < 2
WHERE s.name = 'dw'
GROUP BY s.name, t.name
ORDER BY 1;
```

10. Limites, choix discutables et questions probables

Les choix ci-dessous sont défendables, mais il faut savoir les justifier : ce sont les questions qu'une enseignante posera le plus volontiers.

Limites connues

- **Lookup sur la version courante seulement.** Les faits cherchent toujours la version courante de l'employé, et non celle qui était valable à la date du fait. Tant qu'aucun employé n'a changé, c'est identique. Après des changements, un pointage de 2023

rechargé pointerait vers la version actuelle. La correction consisterait à chercher la version avec `date_debut <= date du fait <= date_fin` (Lookup en cache partiel ou jointure SQL).

- **`fact_pointage` est rechargé entièrement** à chaque exécution. C'est simple et sûr pour 273 000 lignes. Avec des millions de lignes, on passerait à un chargement incrémental (par `_BatchId` ou par date).
- **Fêtes religieuses** absentes de `dim_date`, car elles sont mobiles. Il faudrait les saisir dans une table de référence.
- **Compétences des cours :** le séparateur double espace laisse quelques libellés longs (section 6).
- **Le contrôle du `row_hash`** dépend de l'ordre et du format des colonnes : modifier la requête source sans modifier le hash déclencherait de fausses nouvelles versions.
- **Données personnelles :** `dim_employe` contient nom, e-mail, adresse et statut marital. Il faut restreindre l'accès dans Power BI (RLS ou colonnes masquées).

Questions probables et réponses

1. **Pourquoi des clés de substitution plutôt que l'UUID de l'employé ?** Un entier est plus compact et plus rapide pour les jointures. Surtout, le SCD 2 nécessite plusieurs lignes pour un même employé : l'UUID ne peut donc pas être la clé primaire.
2. **Comment détectez-vous un changement d'employé ?** Par le `row_hash` SHA-256 de tous les attributs suivis, calculé à la source et comparé à celui de la version courante dans un Conditional Split.
3. **Pourquoi pas le composant « Slowly Changing Dimension » de SSIS ?** Il compare colonne par colonne et exécute une requête par ligne, ce qui le rend lent. Le Lookup + hash + Conditional Split est le motif recommandé et reste lisible.
4. **Que se passe-t-il si un fait ne trouve pas sa dimension ?** Le Lookup est en « Ignore failure », puis le Derived Column met la clé à -1 (membre Inconnu). Aucune ligne n'est perdue et l'anomalie reste visible dans les rapports.
5. **Pourquoi -1 dans les faits, mais des lignes écartées dans les ponts ?** Dans un pont, deux couples (employé, -1) violeraient la clé primaire, et un lien vers « compétence inconnue » n'apporte aucune information.
6. **Le package peut-il être relancé ?** Oui. Les scripts statiques sont en `NOT EXISTS`, les dimensions n'insèrent que le nouveau, les ponts et `fact_pointage` sont vidés, et les snapshots du jour sont supprimés puis rechargés. La 2e exécution l'a démontré (section 9).

7. **Pourquoi le nettoyage est-il dans les requêtes SQL plutôt que dans des Data Conversion SSIS ?** `TRY_CONVERT` transforme une valeur invalide en NULL sans faire échouer le flux. La requête est aussi plus courte et lisible qu'une dizaine de composants. Les transformations propres au DW (clés, SCD, coûts) restent dans SSIS.
8. **Pourquoi `fact_effectif_snapshot` ne s'additionne-t-il pas dans le temps ?** C'est un snapshot périodique : additionner deux dates compterait deux fois les mêmes employés. En DAX, on prend le dernier snapshot (`LASTDATE`) ou une moyenne.
9. **À quoi sert la junk dimension ?** Elle évite 5 colonnes booléennes dans un fait de 273 000 lignes : une seule clé suffit, et les 32 combinaisons sont générées une fois pour toutes.
10. **Pourquoi `date_debut = 1900-01-01` pour la première version ?** Sans cela, une version créée en 2026 ne couvrirait pas les pointages de 2023 si l'on cherchait la version valide à la date du fait.
11. **D'où viennent les taux 35 / 25 / 22 ?** Du fichier `extrainfos.txt` , chargé dans `dim_taux_horaire` en SCD 2 : si un taux change, l'ancien reste disponible pour les calculs historiques.
12. **Comment savoir quand une ligne a été chargée et d'où elle vient ?** Par les colonnes d'audit : `etl_load_date` (exécution du DW), `src_load_date` , `src_file` et `src_batch_id` (chargement du staging et fichier d'origine).

11. Répartition pour la soutenance et marche à suivre

Chaque membre peut porter un bloc du package et en maîtriser les composants ; tout le monde doit cependant pouvoir expliquer les sections 3 et 9.

Membre	Partie à présenter	Sections	Ce qu'il doit savoir montrer dans SSDT
1	Architecture, connexions, variables, ordre d'exécution	2, 3	Le Control Flow, les 4 conteneurs, les 2 connection managers, la variable `SnapshotDateOverride`
2	Initialisation : membres inconnus, `dim_date` , `dim_heure` , junk dimensions	4	Le SQL des 4 tâches et la clause `NOT EXISTS`

Membre	Partie à présenter	Sections	Ce qu'il doit savoir montrer dans SSDT
3	Dimensions SCD 2 (employé, département, taux)	5.1	`DFT_DIM_EMPLOYE` : Lookup, Conditional Split, Multicast, OLE DB Command
4	Dimensions SCD 1 et tables de pont	5.2, 5.3, 6	`DFT_DIM_COURS` (mise à jour) et l'éclatement des compétences
5	`fact_pointage` et règles d'assiduité	7.1, 8	La requête source, les 4 Lookups, le calcul du coût des heures sup
6	`fact_effectif_snapshot`, `fact_formation`, contrôles qualité	7.2, 7.3, 9	Les coûts, les clés -1 expliquées, la preuve de la 2e exécution

Exécuter le package

1. Exécuter d'abord `Load_Staging_Area.dtsx` si les fichiers sources ont changé.
2. Ouvrir `Load_data_warehouse.dtsx` dans SSDT, puis appuyer sur F5. Tous les conteneurs doivent passer au vert.
3. Lancer la requête de contrôle de la section 9 et comparer avec le tableau des volumes.

Basculer sur le serveur Tailscale `tcp:100.102.67.72,61301`

1. Dans SSDT, en bas du package, double-cliquer sur **OLEDB SA connection**.
2. Server name : `tcp:100.102.67.72,61301`. Authentification : SQL Server, utilisateur `BI` et son mot de passe ; cocher « Save my password ». Base : `RH_Staging_Area`.
3. Faire de même pour **OLEDB DW connection**, avec la base `RH_Data_Warehouse`.
4. Cliquer sur « Test Connection » pour chacune, puis enregistrer.
5. Exécuter `Load_Staging_Area.dtsx` puis `Load_data_warehouse.dtsx` sur ce serveur.

- [ ] Le schéma `dw` doit exister sur le serveur Tailscale avec la même structure que la base locale (17 tables et les contraintes).
- [ ] Le login `BI` doit pouvoir faire `TRUNCATE` sur les tables `dw` (droit ALTER) et `SET IDENTITY_INSERT`. Sans ces droits, les tâches `SQL_Truncate_*` et `SQL_Init_Membres_Inconnus` échoueront.

- [ ] Le mot de passe est chiffré avec la clé de l'utilisateur Windows (`EncryptSensitiveWithUserKey`). Chaque membre du groupe devra donc le ressaisir sur son propre poste.

12. Partager le travail avec Git

Le package, le projet, le paramètre des fichiers sources, le `.gitignore` et ce rapport en PDF sont dans la branche **`main`** sur GitHub (commit `84f9084`, fusionné directement sans pull request). Il reste seulement à donner l'accès aux 5 autres membres (point 1 ci-dessous) ; les points 2 à 4 de l'étape 1 ne sont plus nécessaires.

Élément	Valeur
Dépôt	github.com/OBenAchour/Project-BI-HR
Branche du travail	`feature/load_data_warehouse`
Fichiers ajoutés	`Project BI HR/Load_data_warehouse.dtsx`, `Project BI HR/Project BI HR.dtproj`, `Rapport SSIS — Alimentation du Data Warehouse RH.pdf`
Volontairement non publiés	`.vs/`, `bin/`, `obj/` : fichiers propres à chaque poste Visual Studio

Étape 1 : le propriétaire du dépôt (OBenAchour)

1. **Donner l'accès :** sur GitHub, ouvrir *Settings* → *Collaborators* → *Add people*, puis ajouter le compte GitHub de chacun des 5 membres. Chacun accepte ensuite l'invitation reçue par e-mail.
2. **Ouvrir la pull request :** aller sur la page de création de la PR, choisir `main` comme base, puis cliquer sur *Create pull request*.
3. **Faire relire :** au moins un membre ouvre l'onglet *Files changed* et valide la PR (*Review changes* → *Approve*).
4. **Fusionner :** cliquer sur *Merge pull request*, puis sur *Confirm merge*. Le travail est alors dans `main`.

Étape 2 : chaque membre récupère le projet

Première fois (le projet n'est pas encore sur le poste) :

```
git clone https://github.com/OBenAchour/Project-BI-HR.git
cd Project-BI-HR
git checkout main
```

Projet déjà cloné :

```
git checkout main
git pull origin main
```

Pour voir le travail **avant** la fusion, remplacer `main` par `feature/load_data_warehouse`.

Étape 2 bis : indiquer où sont les fichiers sources sur son poste

Le package `Load_Staging_Area.dtsx` ne contient plus aucun chemin propre à un PC. Ses 7 fichiers sources sont trouvés à partir d'un seul **paramètre de projet**, `DossierSources`, que chaque membre règle une fois pour toutes sur son poste.

Fichier	Utilisé par	Chemin calculé
`emp_data.xlsx`	connexion Excel `emp_data.xlsx`	`DossierSources + "\emp_data.xlsx"`
`administration_data.xlsx`	connexion Excel `administration_data.xlsx`	`DossierSources + "\administration_data.xlsx"`
`Administration_attendance.xlsx`	connexion Excel `administration_attendance.xlsx`	`DossierSources + "\Administration_attendance.xlsx"`
`attendance_dataset_2023.csv`	connexion fichier plat `FF_attendance_tech`	`DossierSources + "\attendance_dataset_2023.csv"`
`Coursera_trainings.csv`	connexion fichier plat `Coursera_trainings.csv`	`DossierSources + "\Coursera_trainings.csv"`
`extrainfos.txt`	connexion fichier plat `FF_extrainfos`	`DossierSources + "\extrainfos.txt"`
`departments.ods`	variable `OdsPath` (lue par le Script Component)	`DossierSources + "\departments.ods"`

Chaque connexion porte une *expression* (onglet *Expressions* de ses propriétés, sur `ExcelFilePath` ou `ConnectionString`) qui recalcule le chemin à chaque ouverture et à chaque exécution.

Ce que chaque membre fait, une seule fois :

1. Dans l'Explorateur de solutions de SSDT, double-cliquer sur **Project.params**.

2. Dans la ligne `DossierSources`, remplacer la valeur `C:\CHEMIN\VERS\Project-BI-HR\PI\DataSet` par le chemin du dossier `PI\DataSet` de **son** clone, sans `\` final (exemple : `D:\Cours\Project-BI-HR\PI\DataSet`). Enregistrer.
3. Dire à Git d'ignorer cette modification locale, depuis le dossier du dépôt :

```
git update-index --skip-worktree "Project BI HR/Project.params"
```

4. Fermer puis rouvrir `Load_Staging_Area.dtsx` : les avertissements « fichier introuvable » disparaissent.

La commande `--skip-worktree` fait que `git status` n'affiche plus `Project.params` comme modifié : on ne peut donc pas committer son chemin par erreur. C'est déjà fait sur le poste d'OBenAchour.

- [ ] **Si un jour on doit vraiment modifier `Project.params` pour tout le monde** (ajout d'un nouveau paramètre), le membre concerné lance d'abord `git update-index --no-skip-worktree "Project BI HR/Project.params"`. Il remet la valeur générique, committe, puis relance `--skip-worktree` et remet son chemin.

Étape 3 : chaque membre exécute le package sur son poste

- [ ] Installer Visual Studio 2022 avec l'extension *SQL Server Integration Services Projects*, ainsi que le pilote *Microsoft OLE DB Driver for SQL Server* (MSOLEDBSQL).
- [ ] Se connecter à Tailscale avec un compte membre du réseau qui donne accès à `100.102.67.72`.
- [ ] Ouvrir `Project BI HR/Project BI HR.sln`.
- [ ] **Ressaisir le mot de passe du login `BI`** dans `OLEDB SA connection` et dans `OLEDB DW connection` (double-clic → *Save my password* → *Test Connection*). Le mot de passe est chiffré avec la clé Windows de celui qui l'a enregistré (`EncryptSensitiveWithUserKey`) : sur un autre poste, SSDT affiche d'abord une erreur de déchiffrement, ce qui est normal. Le mot de passe se transmet de vive voix, jamais par Git.
- [ ] Exécuter `Load_Staging_Area.dtsx`, puis `Load_data_warehouse.dtsx` (F5), puis la requête de contrôle de la section 9.

Les 6 membres travaillent sur le **même** serveur : une seule exécution suffit pour que tout le monde voie les données. Il est inutile que chacun relance le chargement, et deux exécutions simultanées se gêneraient (TRUNCATE pendant une insertion).

Règles de travail à 6

1. **Une branche par tâche**, créée depuis `main` à jour : `git checkout main`, `git pull`, puis `git checkout -b feature/<nom-de-la-tache>`.
2. **Un seul membre modifie un même `.dtsx` à la fois.** Un package SSIS est un gros fichier XML : deux modifications en parallèle sont presque impossibles à fusionner. Annoncez au groupe qui travaille sur quel package.
3. **Ne commitez un `.dtsx` que pour un vrai changement.** Ressaisir le mot de passe modifie le fichier (nouveau texte chiffré) sans rien apporter. Dans ce cas, annulez la modification avec `git restore "Project BI HR/Load_data_warehouse.dtsx"` avant de faire `git pull`.
4. **Ne commitez pas `.vs/`, `bin/` ni `obj/`.** N'ajoutez que les fichiers voulus (`git add <fichier>`), jamais `git add .`.
5. **Passez par une pull request** pour arriver dans `main`, relue par un autre membre.

Pour publier une modification :

```
git add "Project BI HR/<package>.dtsx"
git commit -m "feat/<description courte>"
git push -u origin feature/<nom-de-la-tache>
```

- [ ] **Point ouvert :** les dossiers `.vs/`, `bin/` et `obj/` sont déjà suivis par Git depuis les premiers commits, ce qui produira des conflits entre postes. Il faut ajouter un fichier `.gitignore` et retirer ces dossiers du suivi (`git rm -r --cached`), en un seul commit sur `main`.